

HouseHunt – Finding Your Perfect Rental Home

1. INTRODUCTION

1.1 Project Overview

HouseHunt is a comprehensive full-stack web application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js). The system is designed to digitize and streamline the rental property lifecycle by enabling secure interaction between Renters, Owners, and Administrators. The application follows a structured three-tier architecture ensuring scalability, maintainability, and modular development practices.

1.2 Purpose

The primary objective of HouseHunt is to build a trustworthy digital rental ecosystem. The platform eliminates dependency on intermediaries by providing direct interaction between tenants and property owners, while ensuring regulatory oversight through an administrative approval mechanism. The system emphasizes transparency, security, and efficiency in rental transactions.

2. IDEATION PHASE

2.1 Problem Statement

Modern rental platforms often suffer from unverified listings, lack of accountability, and poor communication workflows. Renters frequently encounter fraudulent listings, while property owners struggle to reach authentic tenants. This results in mistrust and inefficiency within the rental marketplace.

2.2 Empathy Map Canvas

User Perspective (Renter):

The renter seeks affordable and verified housing options. They often feel anxious about scams and hidden charges. They compare multiple listings, apply search filters, and require transparency before committing to a booking.

2.3 Brainstorming

During ideation, multiple solution approaches were evaluated including decentralized listing systems, AI-based verification, and blockchain validation. After feasibility analysis, the final solution selected was a structured MERN-based platform with admin-controlled verification, role-based dashboards, and secure booking workflow management.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey map

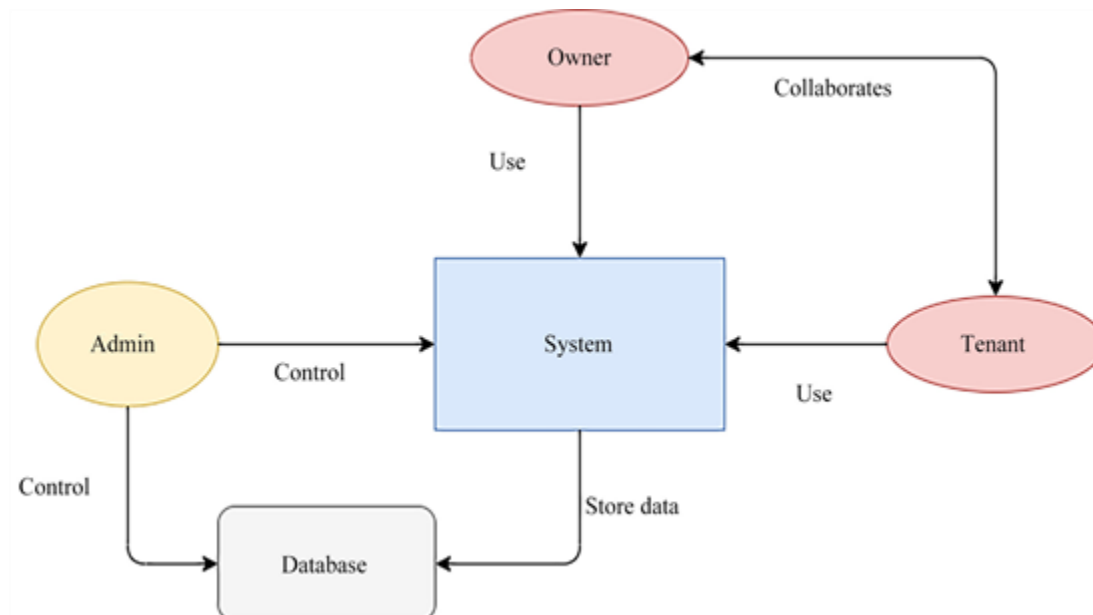
The renter journey begins with account registration and authentication. After successful login, the renter browses properties using advanced filtering mechanisms. Upon selecting a property, the renter submits a booking request, which is reviewed by the owner. Once approved, the booking status is updated and reflected in the renter dashboard.

3.2 Solution Requirement

Functional requirements include secure authentication, property CRUD operations, booking workflow management, and admin approval mechanisms. Non-functional requirements include high availability, secure password hashing, role-based authorization, database scalability, and optimized response time under concurrent load.

3.3 Data Flow Diagram

The Level-0 Data Flow Diagram represents the interaction between user interface, application logic, and database layer. User requests are processed by the React frontend and routed to the Express backend via REST APIs. Data persistence is handled by MongoDB using structured schemas.



3.4 Technology Stack

Frontend technologies include React.js with component-based architecture and Axios for HTTP communication. The backend is implemented using Node.js and Express.js. MongoDB serves as the NoSQL database, while authentication is secured using JSON Web Tokens (JWT) and bcrypt hashing.

4. PROJECT DESIGN

4.1 Problem Solution Fit

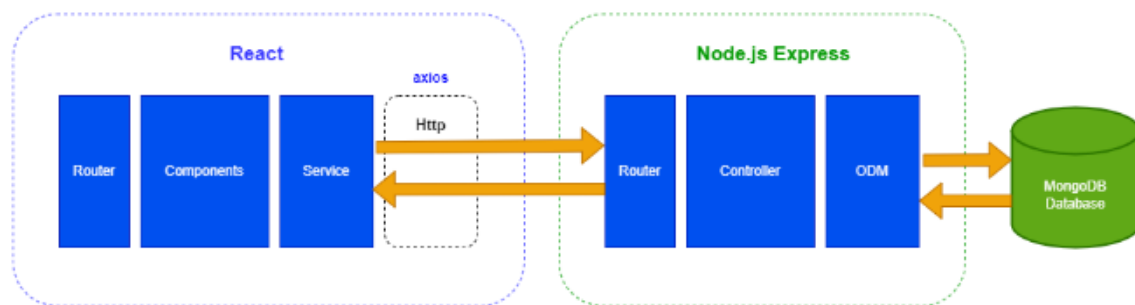
The solution effectively aligns with the identified customer pain points by introducing verified owner onboarding and structured digital workflows. The system directly addresses trust deficits and operational inefficiencies.

4.2 Proposed Solution

HouseHunt operates as a three-tier architecture web application enabling secure rental transactions. The application ensures separation of concerns between presentation, logic, and data layers.

4.3 Solution Architecture

The architecture follows a layered approach. The React-based frontend handles user interaction, the Node/Express backend manages API processing and business logic, and MongoDB handles persistent data storage.



5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Development followed Agile Scrum methodology with sprint-based execution. Sprint 1 focused on authentication, Sprint 2 on property management, and Sprint 3 on booking workflows and admin governance. The team maintained an average velocity of 20 story points per sprint.

6. FUNCTIONAL AND PERFORMANCE TESTING

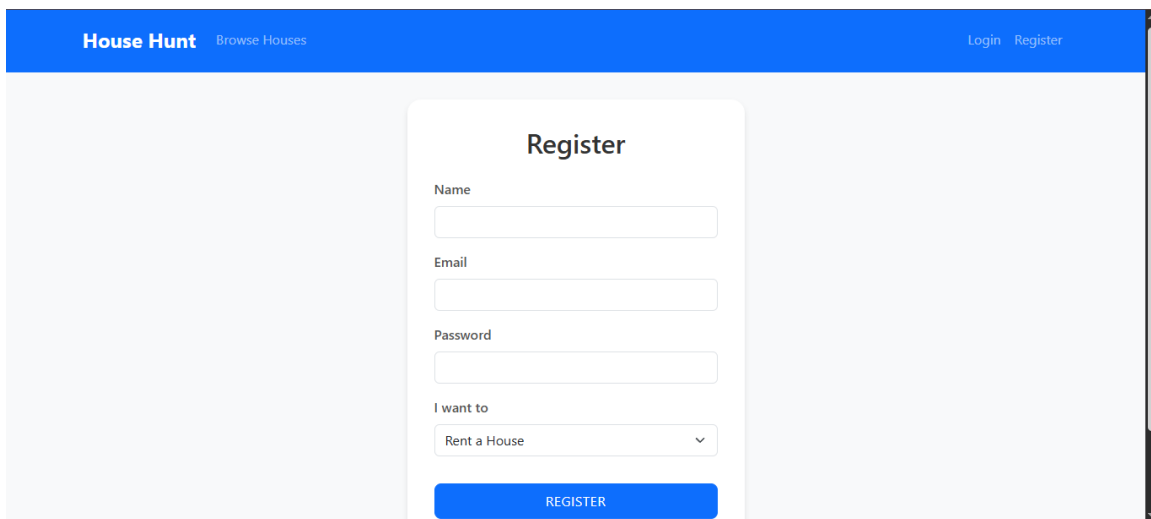
6.1 Performance Testing

The system underwent manual and API-level testing using Postman. Under simulated load of 50 concurrent users, the application maintained average response time of approximately 1.2 seconds, demonstrating stable performance.

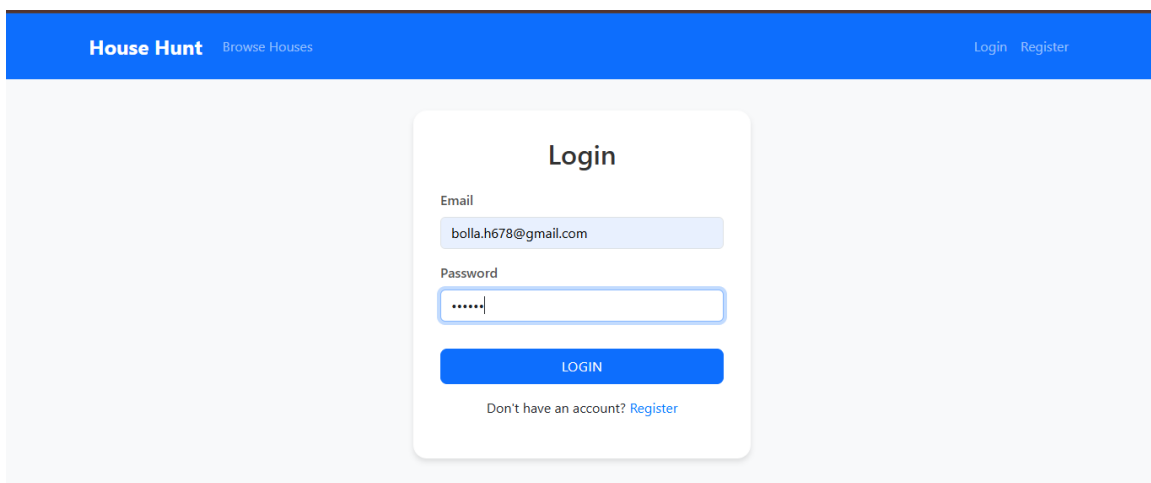
7. RESULTS

7.1 Output Screenshots

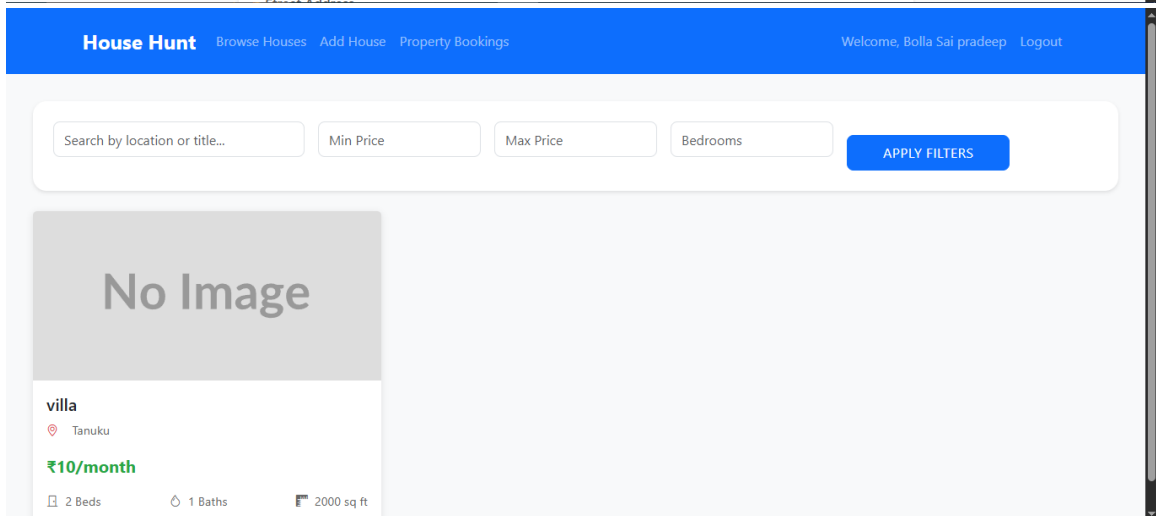
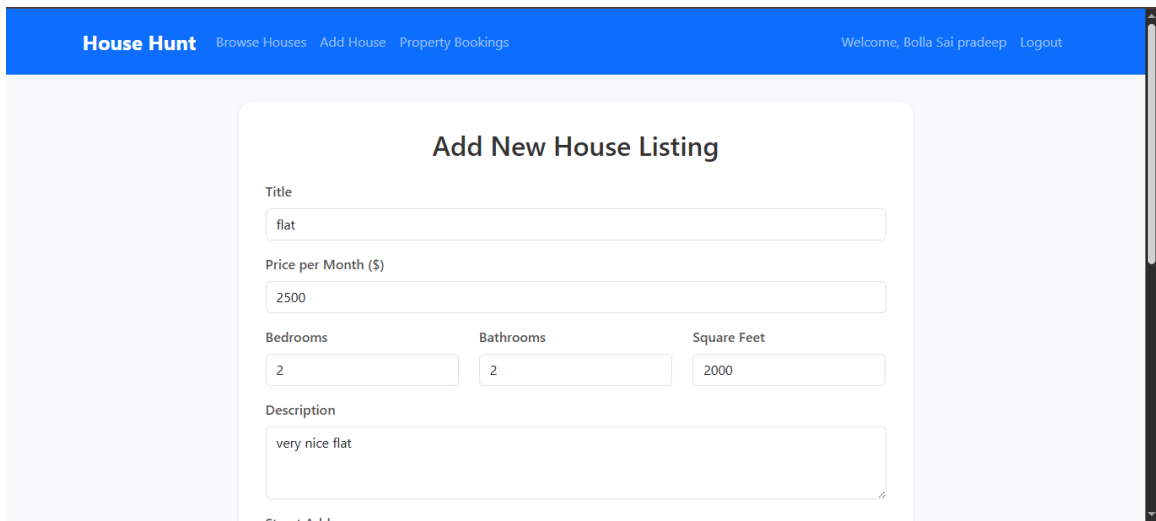
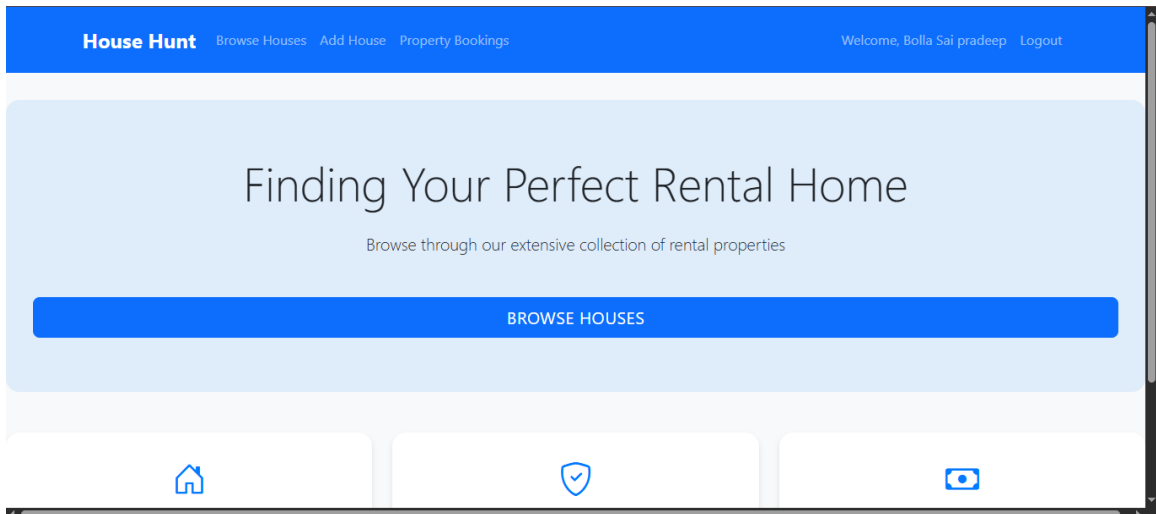
The application successfully demonstrated complete workflow execution including registration, login, property listing, booking submission, and admin approval mechanisms.

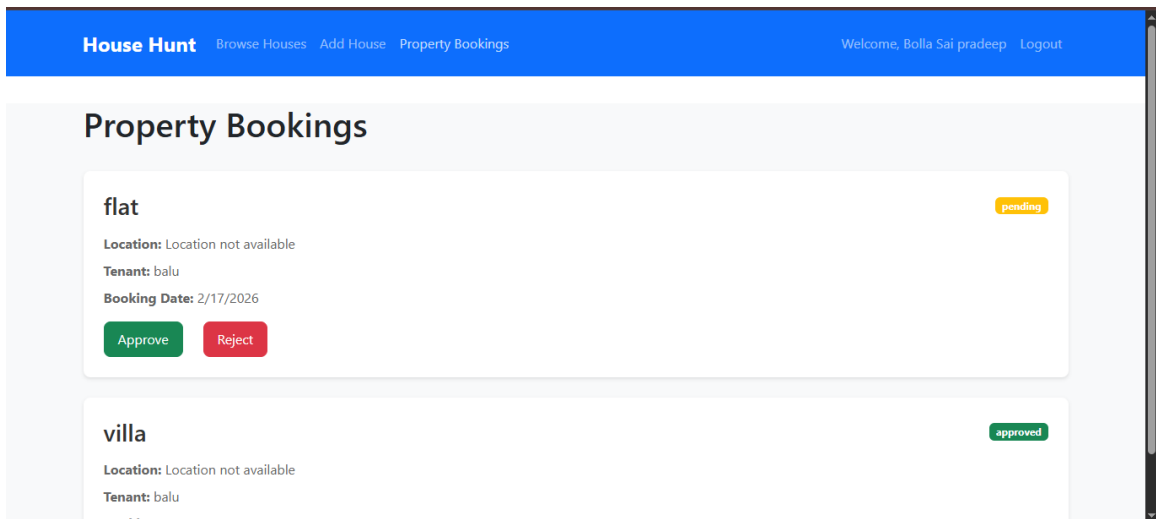
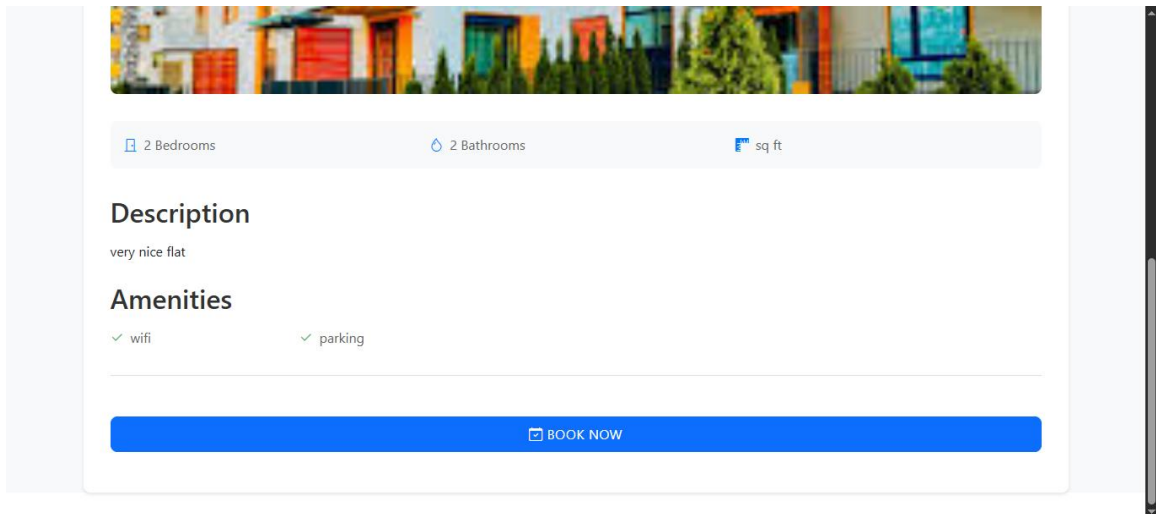


The screenshot shows the 'Register' form in the 'House Hunt' application. The form is centered on a light gray background. At the top, there is a blue header bar with the text 'House Hunt' and 'Browse Houses' on the left, and 'Login Register' on the right. The form itself has a white background and a blue border. It contains the following fields: 'Name' (text input), 'Email' (text input), 'Password' (text input), and 'I want to' (dropdown menu with 'Rent a House' selected). Below these fields is a blue button labeled 'REGISTER'.



The screenshot shows the 'Login' form in the 'House Hunt' application. The form is centered on a light gray background. At the top, there is a blue header bar with the text 'House Hunt' and 'Browse Houses' on the left, and 'Login Register' on the right. The form itself has a white background and a blue border. It contains the following fields: 'Email' (text input with 'bolla.h678@gmail.com' entered) and 'Password' (text input with masked characters '.....'). Below these fields is a blue button labeled 'LOGIN'. At the bottom of the form, there is a link that says 'Don't have an account? Register'.





8. ADVANTAGES & DISADVANTAGES

Advantages include secure authentication, scalable architecture, modular backend design, and role-based access control. Limitations include absence of integrated payment gateway and lack of real-time communication module.

9. CONCLUSION

HouseHunt represents a secure and scalable rental management system built using modern full-stack technologies. The system meets core functional requirements and demonstrates extensibility for future enhancements.

10. FUTURE SCOPE

Future enhancements may include payment gateway integration, AI-based property recommendations, real-time chat modules, and deployment to scalable cloud infrastructure.

11. APPENDIX

Source Code: <https://github.com/pradeep4129d/HouseHunt>

demo video:

<https://drive.google.com/file/d/144QLKHteG6EORhQgW5xNwePHra8D4Vbi/view?usp=sharing>